ARAÇ KİRALAMA SİTESİ

NEDEN ARAÇ KİRALAMA SİTESİNE İHTİYACIMIZ VARDIR

1. Kullanım Kolaylığı ve Hız

- **Hızlı İşlem**: Fiziksel bir ofise gitmeye gerek kalmadan, internet üzerinden hızlıca araç kiralayabilirsiniz.
- **Online Rezervasyon:** Birkaç tıklama ile araç seçeneklerini görebilir, rezervasyon yapabilir ve ödeme işlemlerini gerçekleştirebilirsiniz.

2. Geniş Araç Seçenekleri

- **Çeşitlilik**: Ekonomik araçlardan lüks araçlara, SUV'lardan minibüslere kadar geniş bir araç yelpazesi.
- **İhtiyaca Uygunluk**: Farklı ihtiyaçlara ve bütçelere uygun araç seçenekleri.

NEDEN ARAÇ KİRALAMA SİTESİNE İHTİYACIMIZ VARDIR

3. Fiyat Karşılaştırma

- **En İyi Teklifler**: Farklı araç kiralama şirketlerinin fiyatlarını karşılaştırarak en iyi teklifi bulma imkanı.
- **Şeffaflık**: Aynı platform üzerinden çeşitli teklifleri görerek bilinçli bir seçim yapma.

4. Kampanyalar ve İndirimler

- **Özel Fırsatlar**: Zaman zaman sunulan özel indirimler ve kampanyalar.
- **Ekonomik Seçenekler**: Daha ekonomik bir kiralama deneyimi.

NEDEN ARAÇ KİRALAMA SİTESİNE İHTİYACIMIZ VARDIR

5. Kullanıcı Yorumları ve Değerlendirmeler

- **Güvenilirlik**: Diğer kullanıcıların deneyimlerini okuyarak güvenilir ve kaliteli hizmet sunan şirketleri seçme imkanı.
- **Bilinçli Karar**: Daha bilinçli bir karar verme süreci.

6. Esneklik ve İhtiyaçlara Uygunluk

- **Esnek Çözümler**: Farklı ihtiyaçlara uygun esnek kiralama seçenekleri.
- **One-Way Kiralama**: Aracı belirli bir noktada alıp başka bir noktada bırakma imkanı.

NEDEN ARAÇ KİRALAMA SİTESİNE İHTİYACIMIZ VARDIR

7. Rezervasyon Yönetimi

- **Kolay Yönetim**: Rezervasyonlarınızı kolayca yönetme, iptal etme veya değiştirme imkanı.
- **Planlara Uyum**: Değişikliklere hızlı ve kolay uyum sağlama.

8. Ek Hizmetler

- **Opsiyonel Hizmetler**: GPS cihazı, bebek koltuğu, ek sürücü gibi ek hizmetler.
- **Konfor ve Güvenlik**: Seyahatinizi daha konforlu ve güvenli hale getirme.

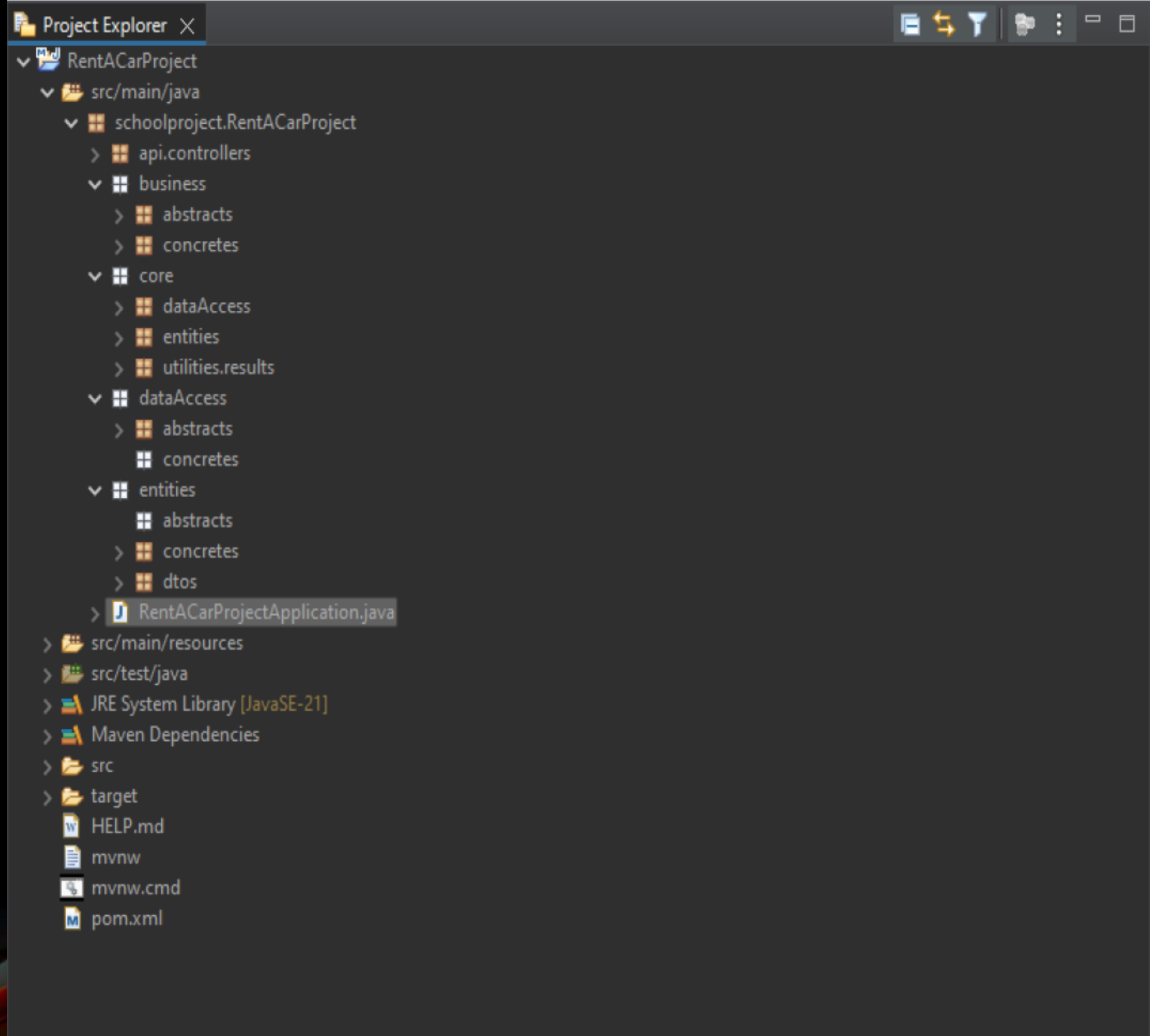
KULLANILAN TEKNOLOJİLER

Frontend: HTML, CSS, JavaScript, React

Backend: Spring Boot, Java

Veritabanı: PostgreSQL

PROJE MİMARİSİ



PROJE MİMARİSİ KISIMLARI

- **Entity :** Varlıkların oluşturulduğu katmandır. Projeye ait gerekli, veritabanında tablosu oluşturulacak tüm varlıklar burada tanımlanır.
- **Data Access :** Veritabanı işlemlerinin yapıldığı katmandır. Oluşturduğumuz tabloların özelliklerini hangi şartlarda getireceğimiz ve API'ye nasıl ileteceğimiz burada belirtilir.
- **Core :** Evrensel kodlamaların yapıldığı bir katmandır. Bu katmanda yazılan kodlar projeye özgü olmaz ve başka projelerde de kullanılabilir.
- **Business :** İş kodlarının yazıldığı katmandır. Varlıkları ekleme, çıkarma, güncelleme ve bir sürü diğer işlemleri hangi kurallara göre yapacağımızı belirleriz.
- **API :** Burada yazdığımız kodların dış dünya ile ilişkisinde kapı görevi gören bir arayüzdür.

KATMANLAR

1)ENTITY KATMANI

```
@Data
@Entity
@Table(name = "cars") //Hangi tabloya karşılık geldiğini yazıyoruz.
@AllArgsConstructor //Parametreli constructor için
@NoArgsConstructor //Parametresiz constructor için
public class Car {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY) //Birer birer arttığını
gösterir.
@Column(name = "id")
private int id;

//@Column(name = "brand_id")
//private int brandId;

//@Column(name = "model_id")
//private int modelId;

//@Column(name = "color_id")
//private int colorId;

@Column(name = "description")
private String description;

@Column(name = "model_year")
private int modelYear;

@Column(name = "daily_price")
private double dailyPrice;

@ManyToOne()
@JoinColumn(name = "brand_id")
private Brand brand;

@ManyToOne()
@JoinColumn(name = "model_id")
private Model model;

@ManyToOne()
@JoinColumn(name = "color_id")
private Color color;

}
```

1)ENTITY KATMANI

Bu Entity katmanından “Car” sınıfına örnektir. Spring Boot kurmamız sayesinde çoğu veritabanı işlemini kodla yazmak yerine bir etiket ile halledebildik. Öncelikle Araba’ya ait gerekli özellikleri tanımladık. En üstte Lombok kurmamız sayesinde gelen bir özellikle @Data etiketi tanımladık ve bu da getter-setter işlemlerini direkt yazmamıza gerek olmadan yapmayı sağladı. @AllArgsConstructor ve @NoArgsConstructor etiketleriyle parametreli ve parametresiz constructorları tanımladık. @Table etiketi ile bu sınıfın veritabanındaki “cars” tablosuna karşılık geldiğini belirttik. Diğer özelliklerin üzerindeki @Column etiketi de o tablonun satırlarına karşılık geliyor. brand_id, model_id ve color_id bu araba tablosunda foreign_key görevi görmektedir. Çünkü bu üçü de ayrı bir tabloya sahiptir. Bu id’leri düz bir şekilde tanımlamayıp sınıflarından nesne oluşturarak getirdik ve isim olarak id’lerine eşitledik. @JoinColumn ile birden fazla tabloyu birleştirmiş olduk.

2)DATA ACCESS KATMANI

```
public interface CarDao extends JpaRepository< Car , Integer> {
 Car getByDescription(String description);
 Car getById(int id);
 Car getByModelYear(int modelYear);
 Car getByDailyPrice(double dailyPrice);
 @Query("SELECT c FROM Car c WHERE c.id = :id AND c.brand.id = :brandId")
 List<Car> getByIdAndBrand(@Param("id") int id, @Param("brandId") int
brandId);
 @Query("SELECT c FROM Car c WHERE c.id = :id AND c.model.id = :modelId")
 List<Car> getByIdAndModel(@Param("id") int id, @Param("modelId") int
modelId);
 @Query("SELECT c FROM Car c WHERE c.id = :id AND c.color.id = :colorId")
 List<Car> getByIdAndColor(@Param("id") int id, @Param("colorId") int
colorId);
 List<Car> getByModelIn(List<Integer> models);
 List<Car> getByColorIn(List<Integer> colors);
 List<Car> getByBrandIn(List<Integer> brands);
 List<Car> getByDescriptionContains(String description);
 List<Car> getByDescriptionStartsWith(String description);
 @Query("SELECT new
schoolproject.RentACarProject.entities.dtos.CarWithBrandDto(c.id, b.id,
c.description, b.name, c.dailyPrice) FROM Brand b INNER JOIN b.cars c")
 List<CarWithBrandDto> getCarWithBrandDetails();
 @Query("SELECT new
schoolproject.RentACarProject.entities.dtos.CarWithColorDto(c.id, co.id,
c.description, co.name, c.dailyPrice) FROM Color co INNER JOIN co.cars c")
 List<CarWithColorDto> getCarWithColorDetails();}
```

2)DATA ACCESS KATMANI

CarDao sınıfımız JpaRepository'den implement edilmiştir. Bu Spring Boot tarafından gelmektedir. Bu sayede tüm ekleme, çıkarma gibi işlemleri arka alanda yapabiliyor. Biz burada veritabanı ile ilgili işlemleri uyguluyoruz. Bunları kodumuzu çalıştırdıktan sonra Swagger sitesinde deneyebiliyoruz. Verilerimize ekleme de yapabilmekteyiz. Birden fazla tabloyu birleştirip verileri döndürebilmek için de @Query etiketiyle özel SQL sorguları yazıyoruz. Sadece tek bir veri döndürmediklerimizi 'List' ile döndürüyoruz.

PROJEDE NEDEN INHERITANCE VE INTERFACE KULLANDIK

Inheritance:

- JPARepository'deki özelliklere DataAccess katmanındaki sınıfların da erişebilmesi ve kullanabilmesi için "Kalıtım" özelliğini kullandık. Core katmanında da Result'ın sahip olduğu success ve message özellikleri, inherit edilen sınıflara iletilmiş oldu.

Interface:

- Business katmanında açtığımız Service Interface'lerine belirli davranışlar atadık ve Manager sınıflarının bu davranışları uygulamasını sağladık. Ayrıca katmanların birbiriyle ilişkili olabilmesi için Interface kullanmamız şarttır.

3)CORE KATMANI

```java
public class Result {
private boolean success;
private String message;

public Result (boolean success) { //Sadece true/false döndürmek
isteyebiliriz.
this.success= success;
}

public Result (boolean success, String message) {
this(success);
this.message = message; //ilgili nesneleri yapılandırmış olduk.
}
public boolean isSuccess() { //Dışarıdan erişim için uygulandı.
return this.success;
}
public String getMessage() {
return this.message;
}
}
```

3)CORE KATMANI

Core katmanında Result sınıfı oluşturduk. Bu veri işlemlerinde olumlu olumsuz dönüş almamız için uygulanmıştır. Message ve success tanımladık. Hem message hem de success döndürmek istiyorsak veya message döndürmek istemiyorsak onlar için ayrı ayrı constructor oluşturduk. İki farklı metot oluşturarak message ve success nesnelerini return ediyoruz.

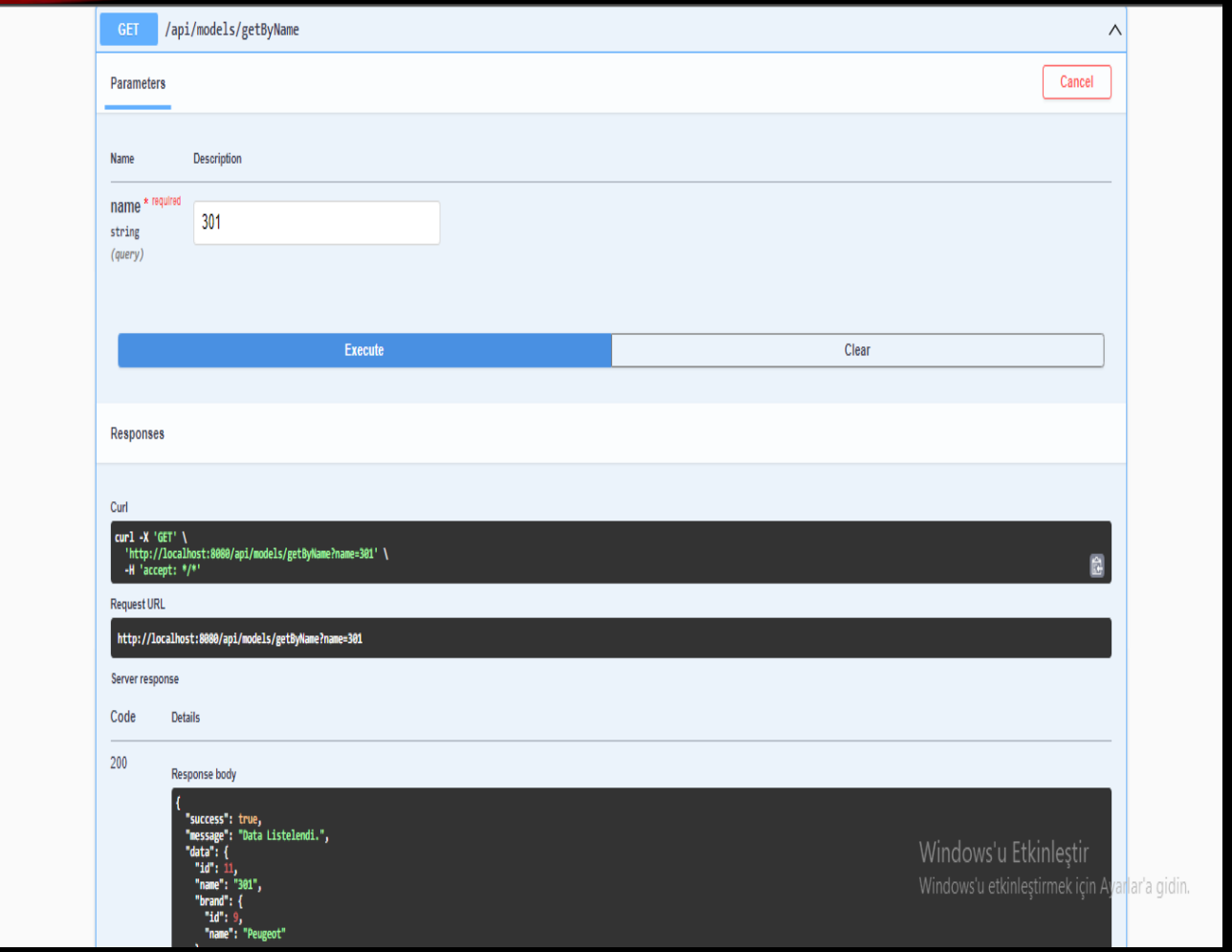
3)CORE KATMANI

```
public class SuccessResult extends Result {
public SuccessResult() {
super(true);
}


public SuccessResult(String message) {
super(true,message);
}
}
```

3)CORE KATMANI

Bir SuccessResult oluşturduk ve Result dan inherit edildi. Böylece oradaki tüm özellikleri buraya tanımladık. True/False dönüşünü kesinlikle true olmaya ayarladık ve mesaj veya mesajsız dönüşü için iki farklı constructor oluşturduk. “super” komutuyla base sınıfın(Result) constructor’larını çalıştırıyoruz. Aynı şekilde bir tane de ErrorResult oluşturup oraya da true/false dönüşünü false olarak ayarladık. Bu sayede Business katmanındaki ve Controller katmanındaki işlemlerde olumlu dönüşleri direkt bu sınıflarla tanımlayabiliyoruz. Ayrıca eğer bir List<Car> döndürülecekse bunlar için de DataResult tanımlandı.



Core katmanındaki yazdığımız kodların kullanım örneği soldaki görüntüde gözükmektedir.Swagger'da API işlemleri denediğimizde Result sınıfında tanımladığımız "success" ve "message" burada true ve Data Listelendi olarak geri dönüş sağlıyor. Bu da kullanıcının kodunun doğru çalışıp çalışmadığına yönelik kolaylık sağlamaktadır.

CONSTRUCTOR VE THIS METHODU

Constructor:

- Constructor oluşturmamız sayesinde nesne oluşturduğumuzda ilk bu metotun çalışmasını sağlıyoruz. Başlangıç değeri atamamızı da sağlıyor. Ayrıca diğer inherit ettiğimiz sınıflarda kullandığımız super methodu ile bu yapıcı metodun çalışabilmesine de olanak sağlıyor.

This:

- Oluşturduğumuz constructordaki parametrelere başlangıç değeri atayabilmek için this methodunu kullandık. Core katmanında birden fazla yapıcı metot oluşturduk ki kullanıcı hangi dönüş tipini istiyorsa o constructor çalışsın ve onun değerleri atansın.

4)BUSINESS KATMANI

```java
@Service
public class CarManager implements CarService {

private CarDao carDao;

@Autowired
public CarManager(CarDao carDao) {
super();
this.carDao = carDao;
}

@Override
public DataResult<List<Car>> getAll() {
return new SuccessDataResult<List<Car>>
(this.carDao.findAll(),("Data Listelendi."));
}

public Result add(Car car) {
this.carDao.save(car);
return new SuccessResult("Araba Eklendi.");
}

public DataResult<List<Car>> getByIdAndBrand(int id, int brandId) {
return new SuccessDataResult<List<Car>>
(carDao.getByIdAndBrand(id, brandId),"Data Listelendi");
}
```

4)BUSINESS KATMANI

İş kodlarımız bu katmanda görülmektedir. DataAccess'de yazdığımız SQL sorgularını "Service" interface sınıflarında uygun metotlara dönüştürdük. Manager sınıfları, implement etmeleriyle birlikte zorunlu davranışları uygulamak zorunda kaldı.Kod örneğinde görüldüğü üzere ekleme yapıyorsak ona uygun SuccessResult döndürürken, listeleme gerektiren metot uyguluyorsak SuccessDataResult döndürdük. @Autowired anatasyonu ile Spring Boot, Dao sınıflarına karşılık gelebilecek sınıflar üretiyor. @Service ile de bu sınıfın amacını Spring Boot'a iletiyoruz.

5)API KATMANI

```
    @GetMapping("/getall") // api/cars/getall
public DataResult<List<Car>> getAll() {
return this.carService.getAll();
}
@PostMapping(value = "/add")
public ResponseEntity<?> add(@Valid @RequestBody Car car) {
return ResponseEntity.ok(this.carService.add(car));
}
@GetMapping("/getByDescriptionContains")
public DataResult<List<Car>> getByDescriptionContains(@RequestParam String
description){
return this.carService.getByDescriptionContains(description);
}
@GetMapping("/getByIdAndModel")
public DataResult<List<Car>>
getByIdAndModel(@RequestParam("id") int id,@RequestParam("modelId") int
modelId){
System.out.println(id);
System.out.println(modelId);
return this.carService.getByIdAndModel(id, modelId);
}
```

5)API KATMANI

Bu katmanla dış dünyaya kodlarımızı iletebiliyoruz. Önceki katmanlarda oluşturduğumuz sorguları anlamlandırıyoruz. @GetMapping ile verileri getirme işlemi yapıp @PostMapping ile ekleme yapıyoruz. Yanlarında yazdığımız route isimleri de metotun ismiyle aynı olmalı. @RequestParam ile HTTP isteğinin parametresini alıyoruz. @RequestBody ile HTTP isteğinin gövdesinden gelen veriyi alıyoruz.

Bu katmanda yazdığımız metotlar ve route isimleri, Swagger veya Postman gibi alanlarda API mizi test etmek ve bu kodları Frontend kısmında kullanıp kodumuzu somut bir hale getirmeye katkı sağlıyor.

DEPENDENCY INJECTION

```
private CarService carService;
@Autowired
public CarsController(CarService carService) {
super();
this.carService = carService;
}
```

Bu soldaki kod API katmanından bir örnektir.Bunu yapmamız katmanlar arasındaki iletişimi sağlıyor. Bir Interface sınıfı implement edilemez. Bu yüzden bu yaptığımız yöntem ile CarManager'da uygulanmış kuralları API katmanına bağlayabiliyoruz. CarService buna uygun arkada bir class oluşturur ve Controller sınıfının bu görevleri yapabilmesini sağlar. Ayrıca bu sayede katmandaki tek sınıfa bağımlı olmayıp o katmandaki özellikleri direkt kullanabilir durumda oluruz.

colors-controller

POST /api/colors/add

Parameters

Cancel Reset

No parameters

Request body required

application/json

```
{
  "id": 0,
  "name": "Orange"
}
```

Execute

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/colors/add' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 0,
  "name": "Orange"
}'
```

Request URL

```
http://localhost:8080/api/colors/add
```

Server response

Code Details

200

Response body

```
{
  "success": true,
  "message": "Renk eklendi."
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu,09 May 2024 15:14:50 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.

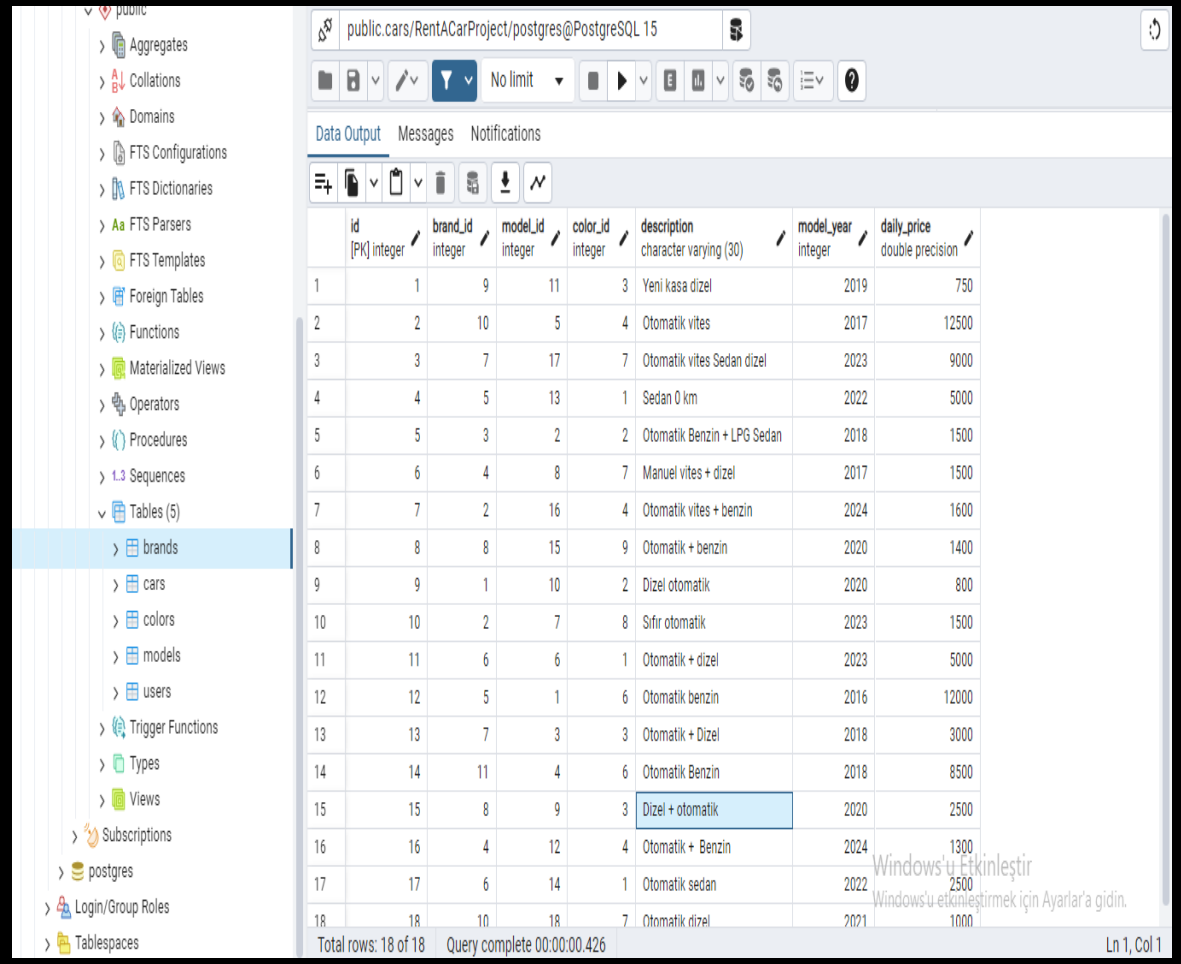
Responses

Code Description Links

```
    },
    {
      "id": 4,
      "name": "Blue"
    },
    {
      "id": 5,
      "name": "Green"
    },
    {
      "id": 6,
      "name": "Yellow"
    },
    {
      "id": 7,
      "name": "Grey"
    },
    {
      "id": 8,
      "name": "Brown"
    },
    {
      "id": 9,
      "name": "Orange"
    }
  ]
}
```


Bu örnekte api/colors/add ile renk ekledik. Tabloya id ve isim yazıp Execute diyerek veritabanı tablomuzda yeni bir renk oluşturduk. Core katmanında oluşturduğumuz Result ve ondan Inherit edilen alt sınıfları API katmanında return tipinde döndürmemiz sayesinde renk eklediğimizde "true" ve "Renk Eklendi" dönüşleri aldık.

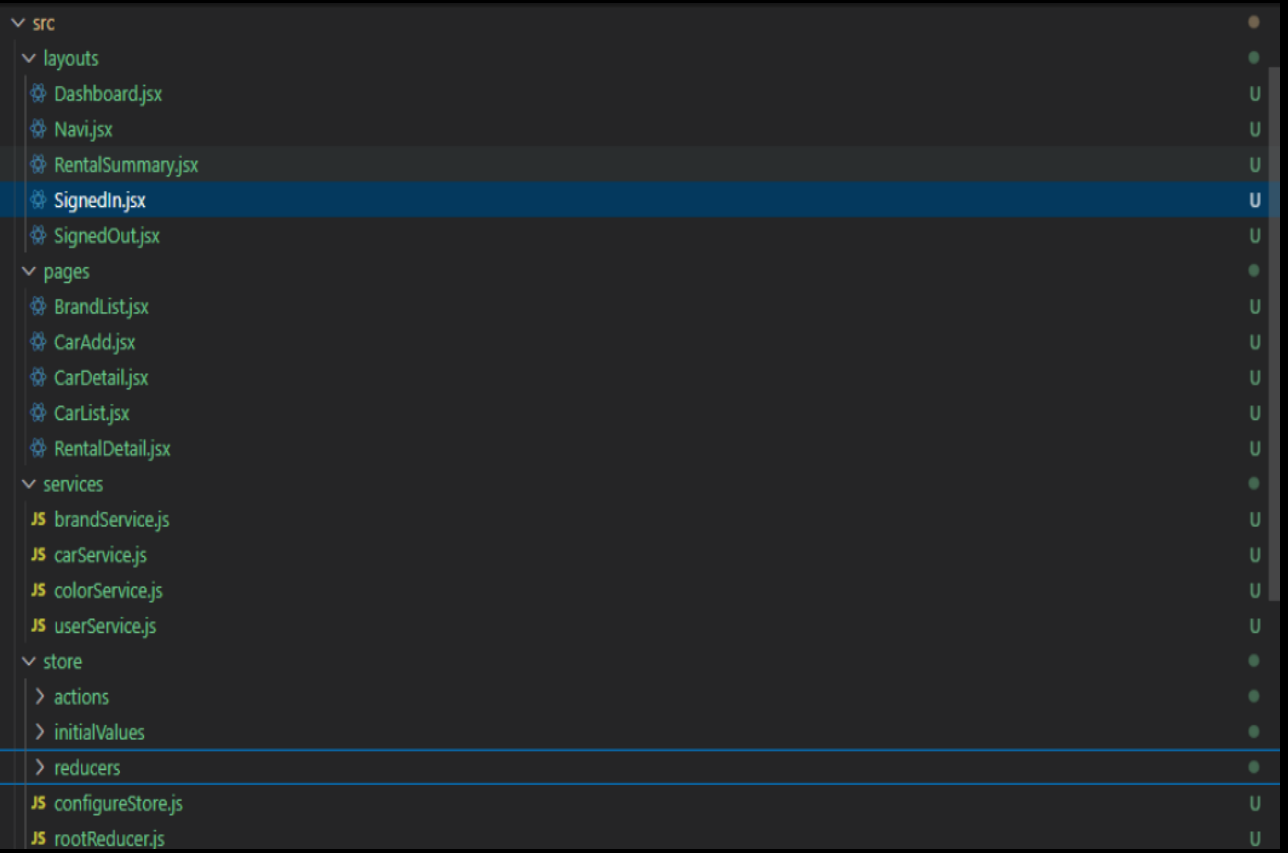
VERİ TABANI YÖNETİM SİSTEMİ



	id [PK] integer	brand_id integer	model_id integer	color_id integer	description character varying (30)	model_year integer	daily_price double precision
1	1	9	11	3	Yeni kasa dizel	2019	750
2	2	10	5	4	Otomatik vites	2017	12500
3	3	7	17	7	Otomatik vites Sedan dizel	2023	9000
4	4	5	13	1	Sedan 0 km	2022	5000
5	5	3	2	2	Otomatik Benzin + LPG Sedan	2018	1500
6	6	4	8	7	Manuel vites + dizel	2017	1500
7	7	2	16	4	Otomatik vites + benzin	2024	1600
8	8	8	15	9	Otomatik + benzin	2020	1400
9	9	1	10	2	Dizel otomatik	2020	800
10	10	2	7	8	Sıfır otomatik	2023	1500
11	11	6	6	1	Otomatik + dizel	2023	5000
12	12	5	1	6	Otomatik benzin	2016	12000
13	13	7	3	3	Otomatik + Dizel	2018	3000
14	14	11	4	6	Otomatik Benzin	2018	8500
15	15	8	9	3	Dizel + otomatik	2020	2500
16	16	4	12	4	Otomatik + Benzin	2024	1300
17	17	6	14	1	Otomatik sedan	2022	2500
18	18	10	18	7	Otomatik dizel	2021	1000

Veritabanı olarak da PostgreSQL kullanıldı. Tabloların verileri burada girildi ve marka, renk ve modeller oluşturuldu. En sonunda da bu id'ler doğru şekilde girilerek arabaları oluşturduk. Bunun yanında açıklama, model yılı ve günlük fiyatlar da girildi.

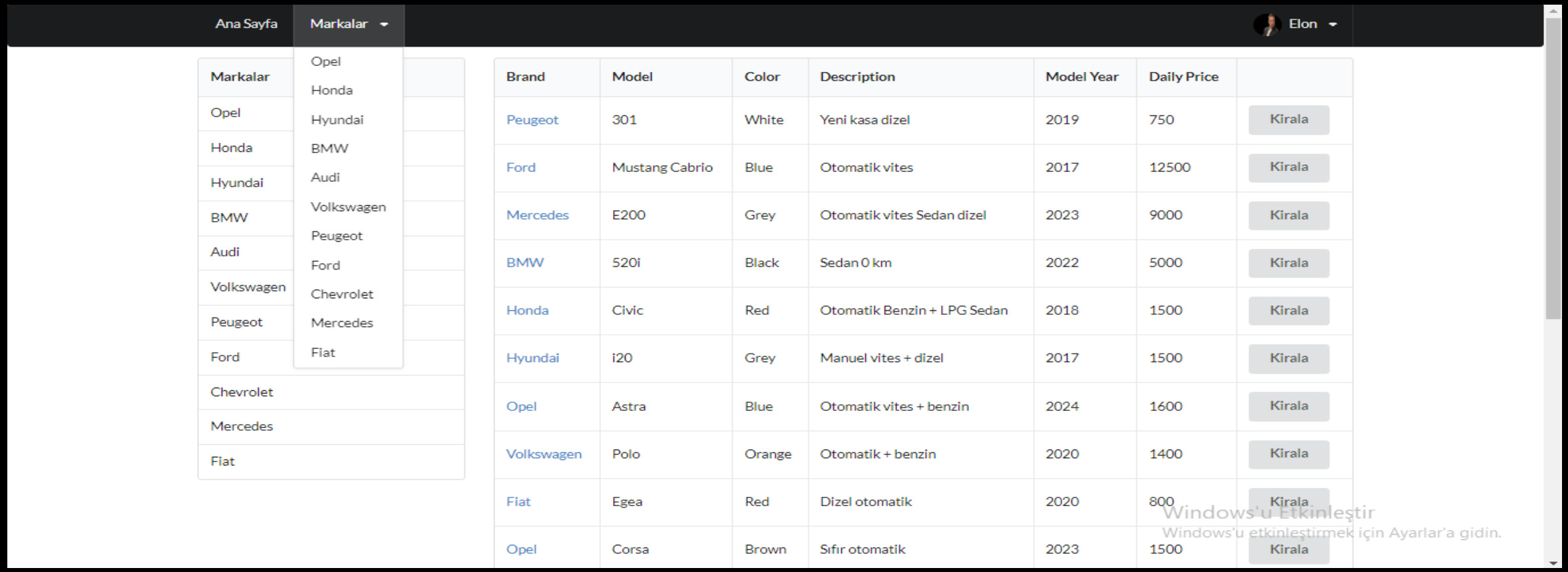
PROJENİN FRONTEND KISMI



Projenin önyüz kısmı için React kullandık. Services dosyasında fonksiyonlar oluşturarak API'den gelen URL'leri kullandık ve verileri çağırdık. Layouts dosyasında ise solda markaları listeleyip yukarıdaki Navbar'ı ekledik.

BrandList ve CarList dosyalarında onlara ait olan Service'leri çağırdık. Map methodu sayesinde o tabloları baştan sona dolaşmasını sağlayıp istediğimiz özelliği veritabanındaki sırası ile yazdırmasını sağladık. Böylece sitemizde oluşturduğumuz tabloyu kendi elimizle doldurmaktan kurtulduk.

WEB SİTEMİZİN GÖRSEL TASARIMI



Kısacası biz bu projede Java programlama dilinde yazdığımız kodlar ile veritabanı ve dış dünya arasında bir ilişki kurulmasını sağladık. Gerekli varlıklar oluşturup onlara başka sınıflarda davranışlar verdik. Birden fazla katman oluşturup hepsine ayrı görevler vererek tek bir sınıftaki kod kirliliğini önledik. Her katmanı anlamlarına uygun kullanarak bir düzenleme yapılacağı zaman zorlanılmasını azalttık. Verileri belirli kurallara koyduk ve kendi içlerinde de iletişim halinde tuttuk. Sonra bu verileri API yardımıyla Frontend kısmına ilettik ve React aracılığıyla bunu bir websitesine dönüştürdük.

HAZIRLAYANLAR

- EMRE YILMAZ
- MUHAMMED NEBİ ALTUN
- SERDAR GÜLER
- YAĞIZ ZORLU